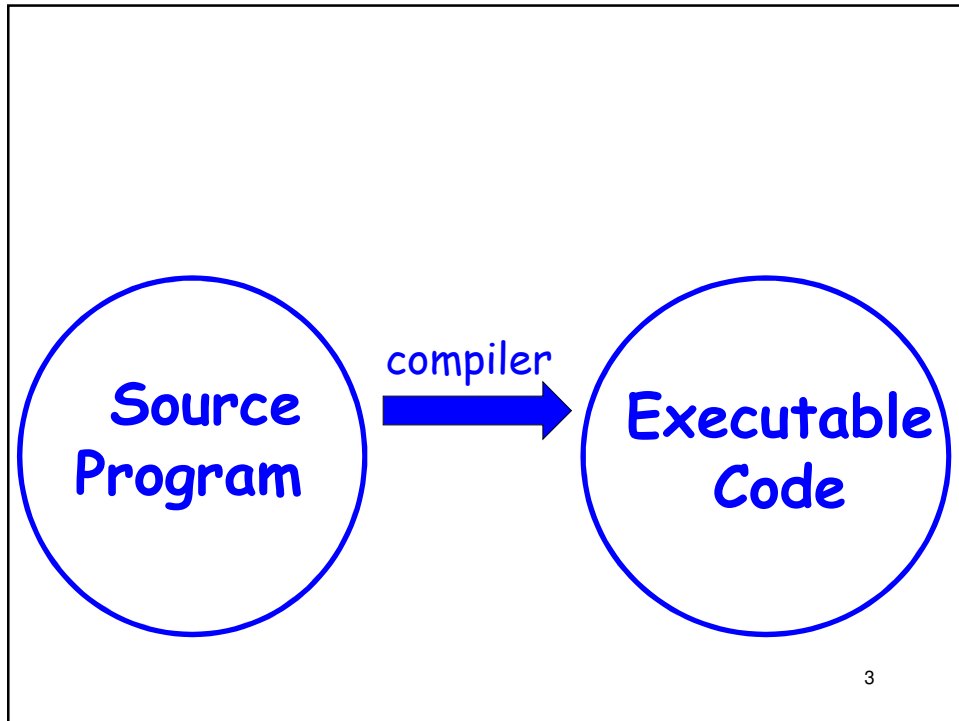


FORMAL LANGUAGES AND COMPILERS

Paola Quaglia

Goals & Expected Learning

- Concepts, techniques, and algorithms for the implementation of compilers
- Formal languages
- Recognition of formal languages



A long way to go

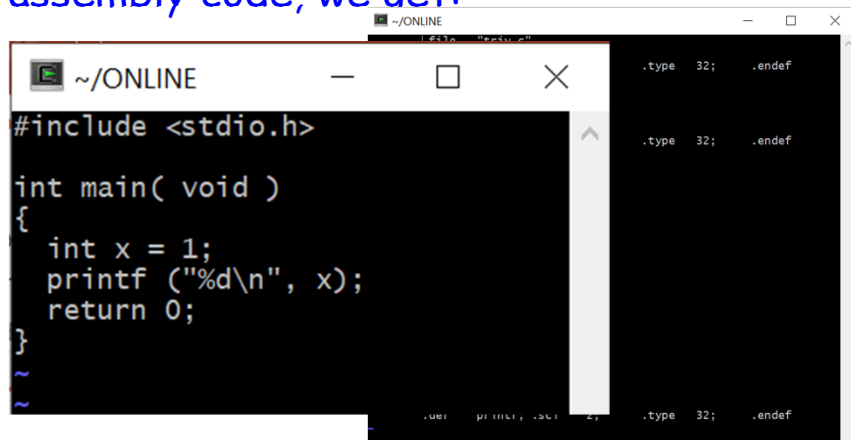
A long way to go

- E.g., the development of gcc has been going on for more than 30 years by now
- Some years ago it already consisted of 2++ million lines of code

5

A long way to go

E.g., from source program to assembly code, we get:



The screenshot shows a code editor window titled '~ / ONLINE' with a dark background. The left pane displays C source code:

```
#include <stdio.h>

int main( void )
{
    int x = 1;
    printf ("%d\n", x);
    return 0;
}

~
~
```

 The right pane shows the corresponding assembly code, with visible lines including `.type 32; .endif` and `.type 32; .endif`. The assembly code is partially obscured by the source code pane.

E.g.,
assembler

```
~ / ONLINE
.file "triv.c"
.text
.def __main; .sc1 2; .type 32; .endif
.LC0:
.ascii "%d\\12\\0"
.text
.globl main
.def main; .sc1 2; .type 32; .endif
.seh_proc main
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rsp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $1, -4(%rbp)
movl -4(%rbp), %eax
movl %eax, %edx
leaq .LC0(%rip), %rcx
call printf
movl $0, %eax
addq $48, %rsp
popq %rbp
ret
.seh_endproc
.ident "GCC: (GNU) 9.3.0"
.def printf; .sc1 2; .type 32; .endif

~ / ONLINE
#include <stdio.h>
int main( void )
{
    int x = 1;
    printf ("%d\\12\\0", x);
    return 0;
}
```

A long way to go

- And assembly code is not yet executable
- Still needed:
 - Conversion of textual instructions to binary instructions
 - Linking to include libraries

Various phases in pipe line

- What we do have:

- The program is written in a language which adheres to a **formal grammar**
- Some programs are syntactically legal, some are not
- The grammar lets us decide on this

9

Various phases in pipe line

Line of code:

`pippo = 1;`

Formal grammar:

An assignment has the form

IDENTIFIER ASSIGN-SYMBOL EXPRESSION SEMICOLON

10

Various phases in pipe line

Another line of code:

```
pluto = 2*3;
```

The same pattern of the formal grammar as before:

An assignment has the form

IDENTIFIER ASSIGN-SYMBOL EXPRESSION SEMICOLON

11

The grammar

- States the form of an assignment
 - IDENTIFIER ASSIGN EXPRESSION SEMICOLON
- Cannot know "pippo", "pluto", nor the infinitely many strings we may use as variables

12

First phase: Lexical analysis

- Translates a stream of characters into a stream of **tokens**
- E.g. translates
- `pluto = 2*3;`
- Into
- `<ID,pluto> ASS <NUM,2> MUL <NUM,3> SEMCOL`
- `ID,ASS,NUM,MUL,SEMCOL` are tokens

13

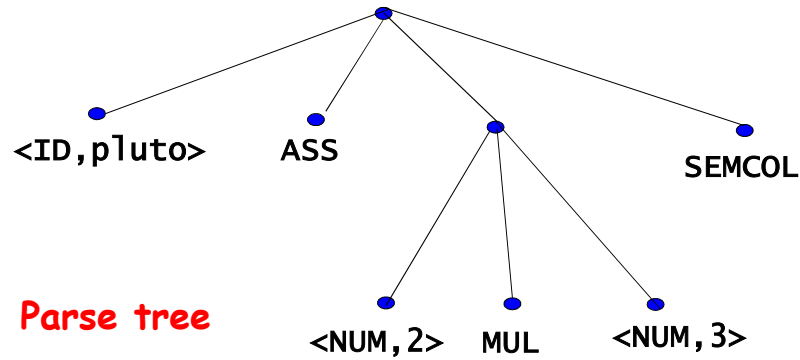
Second phase: Syntax analysis

- Checks whether a stream of tokens like, e.g.,
 - `<ID,pluto> ASS <NUM,2> MUL <NUM,3> SEMCOL`
- Adheres to the given grammar
- If so, converts the stream of tokens into a **parse tree** (or better, into an **abstract syntax tree**)

14

Syntax analysis

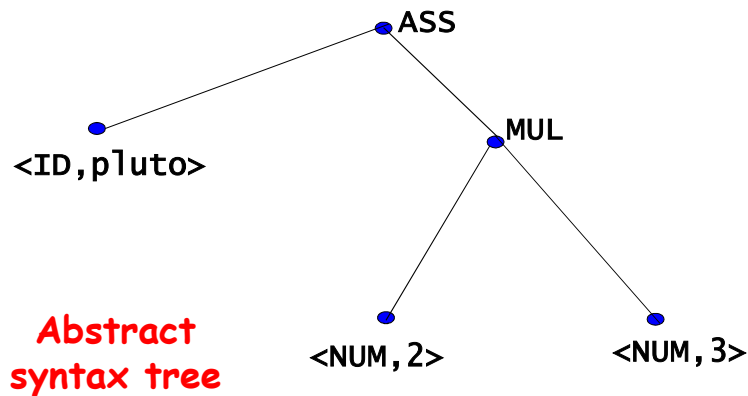
<ID,pluto> ASS <NUM,2> MUL <NUM,3> SEMCOL



15

Syntax analysis

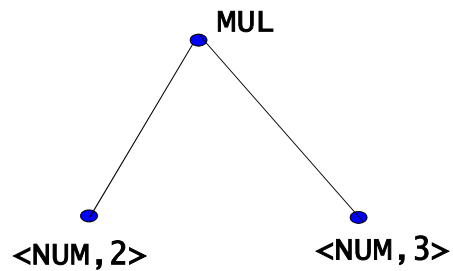
<ID,pluto> ASS <NUM,2> MUL <NUM,3> SEMCOL



16

Semantic analysis

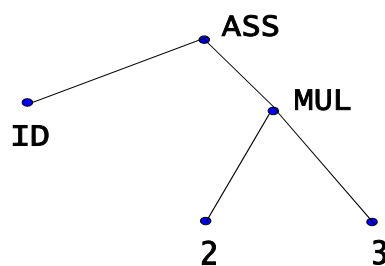
Are we actually multiplying two integers?



17

Intermediate code generation

- Converts a parse tree into a textual intermediate code



```
MOVE 2,R1  
MULT 3,R1  
MOVE R1,ID
```

18

Target code generation

- Translates intermediate code into target machine code (assembly code)



A screenshot of a text editor window titled "triv.c" showing assembly code. The code is for a trivial C program that prints "dr". It includes standard assembly directives like .text, .def, .section, .ascii, .globl, .seh_proc, .seh_pushreg, .seh_setframe, .seh_stackalloc, .seh_endprologue, .seh_endproc, and .ident. The code is written in a syntax that is a mix of C and assembly, with comments in C and assembly instructions in assembly. The code is as follows:

```
.file "triv.c"
.text
.def __main; .scl 2; .type 32; .endif
.section .data,"dr"
LC0:
.ascii "dr"
.text
.globl main
.def main; .scl 2; .type 32; .endif
.seh_proc main:
pushq %rbp
.seh_pushreg %rbp
movq %rsp, %rbp
.seh_setframe %rbp, 0
subq $48, %rsp
.seh_stackalloc 48
.seh_endprologue
call __main
movl $1, -4(%rbp)
movl -4(%rbp), %eax
leaq %eax, %ecx
call LC0(%rip), %rcx
movl $0, %eax
addq %eax, %rsp
popq %rbp
ret
.seh_endproc
.ident "GCC: (GNU) 9.3.0"
.def printf; .scl 2; .type 32; .endif
```

Plausible targets

- Machine code not necessarily the target of compilation
 - May be a virtual machine
 - May be another language

Front-end & Back-end

- Front-end
 - From lexical analysis all the way down to intermediate code generation
- Back-end:
 - All the rest
- Modularity:
 - N languages, N front-ends
 - K machines, K back-ends
 - $N \times K$ compilers, overall

21

That said

22

Goals & Expected Learning

- Pervasive structures (grammars, finite state automata, but also trees and graphs)
- Methodologies at the basis of other areas, too (e.g., parsing of natural languages, soundness of queries to databases)

23

In a Nutshell

- Regular languages and finite state automata (lexical analysis)
- Context-free languages and parsing (syntax analysis)
- Syntax driven definitions (semantic analysis & symbol tables)

24

In a Nutshell (ctd.)

- Intermediate code generation
- Machine code generation
- Optimization techniques

25

Main Textbook

- Compilers: Principles, Techniques, and Tools (2nd edition); Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman; Prentice Hall, 2006

26

Verification

- Written exam

27

Verification

- Subscription on esse3 is compulsory
- Mind the fact that "Inglese B1" is **blocking** for registering LFC
- The test is:
- Closed books & No electronics

28

Test

- 8 questions

- Questions on basics (general knowledge of the subject, theory)
- "Mechanical" questions (application of the algorithms studied in the course to a given structure)
- "Show-off" questions (to show a deeper understanding of the subject)

29

Marking

- For each question

- the answer is correct: 4 points
- no answer at all: 0 points
- the answer is not correct: -1 points

- The score "30 e lode" is given when the total mark of the test is greater than 30

30

"Skip-appello"

- In case of subscription & no-show the student undergoes a skip-appello

31

"Skip-appello"

- If the total mark of the test is less than 10
- The student undergoes a skip-appello

32